

Fintech Lending Platform Stack

Should we adopt this version-locked stack as the production baseline for v1?

RECOMMENDATION: APPROVE WITH CONDITIONS

Discussion output

Approve baseline

Approve with risk controls

Revise blocking choices

Backend

Java 21 + Spring Boot 4.0.6

Data

PostgreSQL 17 + Flyway

Mobile

Expo SDK 55 + React Native 0.83

Ops

VPS + Docker Compose + Nginx

How the stack is judged

A good choice must fit fintech controls and still be simple enough for the team to operate.

Weighted score model



Decision rule

Recommend the highest-scoring option that keeps launch operations understandable.

Scoring scale

- 5 = strong fit now
- 3 = workable with tradeoffs
- 1 = avoid for v1 unless required

Default stance

Approve only when compatibility, security, ops, and observability work are owned.

Version-locked baseline

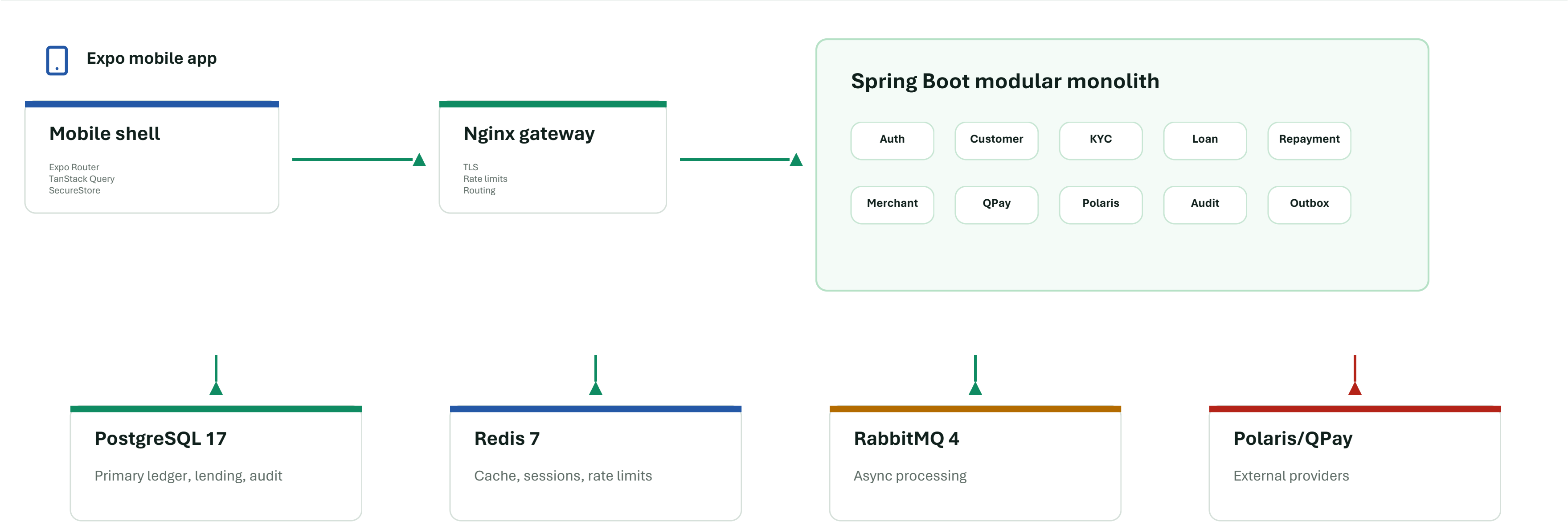
The proposed stack is broad but coherent: one backend runtime, one mobile framework, and simple launch infrastructure.

Platform Java 21 LTS Spring Boot 4.0.6 Gradle 9.x Virtual threads enabled	Backend Web MVC, JPA, Security Validation, Actuator AMQP, Redis, Mail Flyway, Cache, AOP	Data PostgreSQL 17.x Flyway PostgreSQL HikariCP 6.x Hibernate ORM 7.x	Security Spring Security 7.x JWT / JJWT 0.12.6 SecureStore, biometric auth SSL pinning
Resilience Resilience4j 2.4.0 Circuit breaker + retry Apache HttpClient5 5.3.1 Polaris dual logging	Mobile Expo SDK 55 React Native 0.83 Expo Router 4.x TypeScript 5.x	State & API TanStack Query 5.x Zustand 5.x Axios 1.7.x NetInfo, retry, cache	Infrastructure Docker 27 + Compose 2 Nginx 1.26 Redis 7 + RabbitMQ 4 Prometheus/Grafana/Loki

Decision implication: the team optimizes for mature backend controls, fast mobile delivery, and self-managed launch operations.

Modular monolith, clear async boundary

Keep user-facing actions synchronous, push provider sync and notifications into recoverable background workflows.



Solid path: user waits for API response. Background path: outbox -> RabbitMQ -> provider sync -> audit/operation visibility.

Spring Boot modular monolith wins for v1

The backend choice should maximize fintech fit without creating microservice-level operations too early.

Spring Boot modular monolith	Java 21, Boot 4, JPA, Security		4.6	Best balance of controls, delivery, and ops simplicity.
.NET modular monolith	C#, ASP.NET Core, EF Core		4.1	Strong alternative, but current stack is already Java/Spring.
Go services	Go, explicit services		3.6	Fast and lean, but more hand-built fintech plumbing.
NestJS backend	Node.js, TypeScript		3.4	Good velocity, weaker default fit for heavy transactional core.
Microservices first	N services, distributed data		3.1	Scales teams later, increases v1 failure modes now.

Why not microservices now

Loan, repayment, KYC, merchant, and provider workflows need transactional clarity more than independent deployability at launch.

What keeps monolith healthy

Package boundaries, module tests, database ownership rules, and explicit async boundaries around provider work.

Decision

Use Spring Boot modular monolith as v1 production baseline.

The recommendation is strong but not free

Spring Boot is the right default if the team actively manages Boot 4 maturity and module discipline.

Pros

- Mature enterprise and fintech patterns
- Strong JPA, validation, security, and actuator support
- Good fit for transactional workflows and audit trails
- Large hiring and documentation ecosystem
- Virtual threads reduce blocking I/O pressure

Cons

- JVM memory footprint is higher than Go/Node
- Spring Boot 4 library compatibility must be validated
- JPA can hide expensive queries if not reviewed
- Monolith boundaries can blur without enforcement
- Startup/runtime tuning still matters on VPS

Mitigations

- Run dependency and integration smoke tests
- Keep module APIs explicit inside package structure
- Add query logging and performance review gates
- Use outbox/RabbitMQ for provider-side effects
- Set JVM memory and health-check defaults in Docker

APPROVE IF THESE MITIGATIONS BECOME ENGINEERING WORK ITEMS

PostgreSQL 17 is the safest transactional default

Lending records need constraints, migrations, reporting, and JSON flexibility more than schemaless velocity.

Candidate	Fit	Rationale	Decision
PostgreSQL 17	Excellent	Strong constraints, JSONB, reporting, extensions, mature ops	Recommended
MySQL	Good	Solid OLTP, but weaker JSON/query ergonomics for this domain	Viable fallback
MongoDB	Mixed	Flexible documents, weaker fit for lending ledgers and relational constraints	Avoid for core
CockroachDB	Strong later	Distributed SQL, but operational complexity is premature	Revisit at scale

Decision note

Use PostgreSQL as the source of truth. Keep Redis, RabbitMQ, and provider logs as supporting systems, not substitutes for durable lending records.

Two-tier cache plus RabbitMQ keeps v1 fast and operable

Use specialized tools where they reduce real latency or recovery risk; avoid Kafka-level complexity before event streaming is needed.

Cache decision

Caffeine + Redis

Pros: local speed + distributed consistency
Cons: invalidation discipline
Why chosen: best latency/control mix

Redis-only

Pros: simpler shared state
Cons: network hop on hot reads
Why not: misses L1 speed benefit

Memcached

Pros: simple cache
Cons: less versatile than Redis
Why not: no session/rate-limit fit

Recommended
Caffeine + Redis

Queue decision

RabbitMQ

Pros: routing, retries, back pressure
Cons: queue ops ownership
Why chosen: ideal for async jobs

Kafka

Pros: event streaming, replay
Cons: heavy ops model
Why not: premature for v1 scale

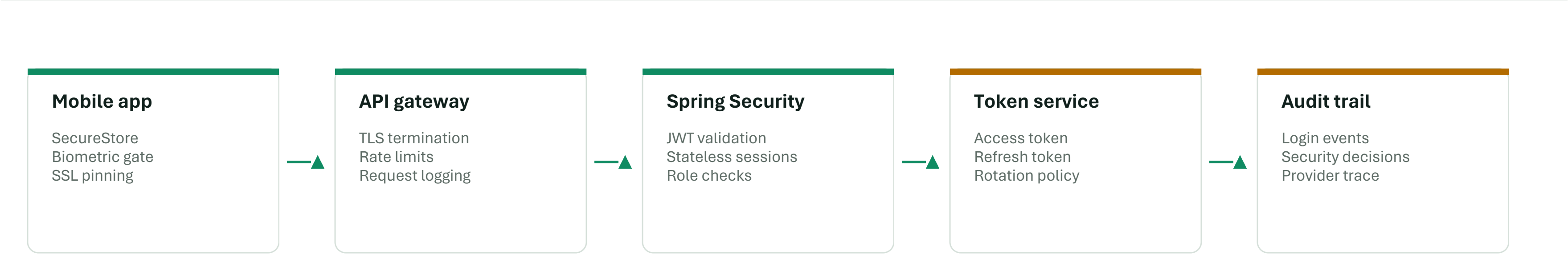
Database jobs

Pros: fewer systems
Cons: weaker worker semantics
Why not: poor fit for provider retry load

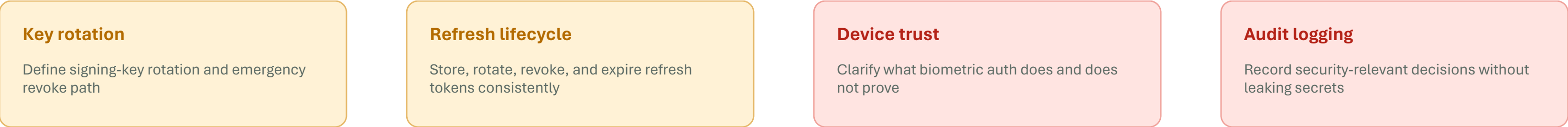
Recommended
RabbitMQ

Strong defaults, explicit hardening work

The stack covers the right primitives, but token lifecycle, key rotation, and mobile device trust must be designed intentionally.

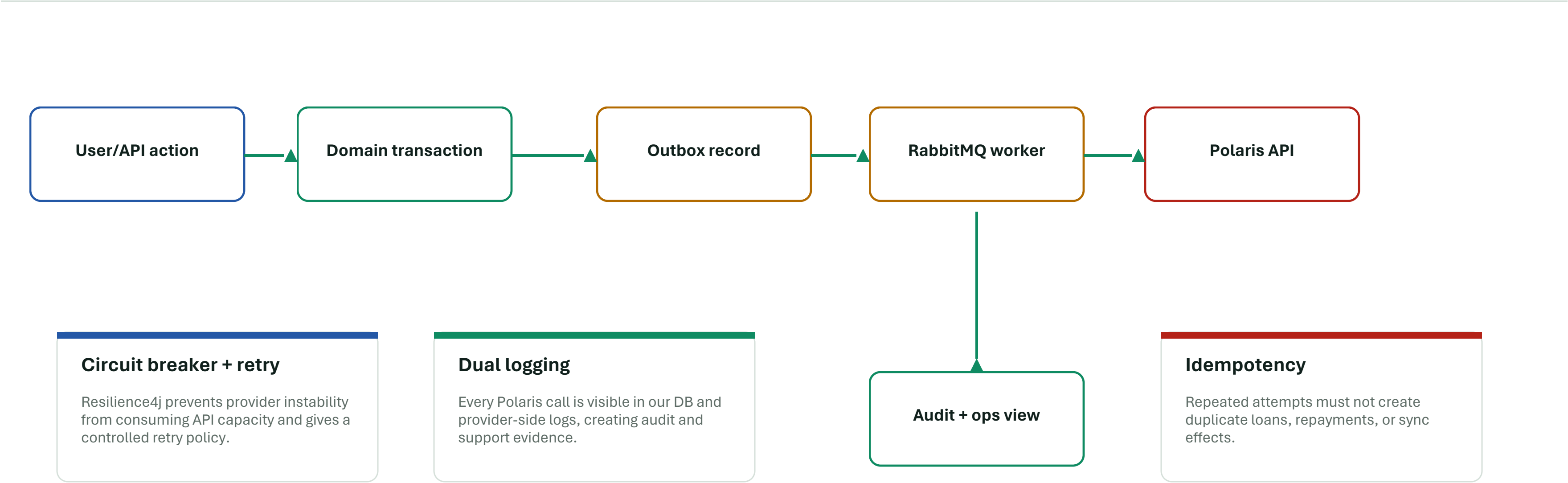


Risks to close before production



Polaris calls must be recoverable and provable

Financial provider failures need idempotency, retries, circuit breakers, and dual logging instead of best-effort API calls.



Recommendation: Resilience4j + outbox/RabbitMQ + dual logging + idempotency keys for all Polaris-facing workflows.

Expo React Native is the fastest credible v1 path

The team gets shared delivery speed and OTA updates, while retaining an escape hatch for native constraints.

Expo React Native		4.5	Fastest v1, OTA updates, shared JS/TS ecosystem	Native-module limits need validation
Bare React Native		4.0	More native control, same RN ecosystem	More setup and release overhead
Flutter		3.7	Strong UI toolkit and performance	Different language/ecosystem from chosen stack
Native iOS/Android		3.2	Maximum platform control	Highest team and delivery cost

Decision: use Expo SDK 55 with React Native 0.83, but validate SSL pinning and compressor/native modules before production release.

VPS + Docker Compose is a deliberate launch choice

The best v1 platform is the one the team can fully understand, secure, monitor, back up, and restore.

VPS + Docker Compose

Pros: low cost, full control, simple mental model
Cons: team owns backups, patches, HA
Why chosen: enough for launch

Kubernetes

Pros: orchestration and scaling primitives
Cons: platform complexity before need
Why not: too much ops surface for v1

AWS managed services

Pros: managed HA options and ecosystem
Cons: cost, lock-in, cloud ops scope
Why not: not aligned with local VPS decision

Serverless

Pros: low server management
Cons: harder for long-running fintech workflows
Why not: awkward for monolith and regulated state

Operating condition

Approval requires a runbook covering deploys, rollback, secrets, OS/container patching, backups, restore tests, monitoring, and incident response.

Open-source monitoring is enough if ownership is explicit

The stack controls cost, but the team must build the dashboards and alerts before real users depend on it.

Recommended baseline

- Prometheus: metrics
- Grafana: dashboards
- Loki: logs
- Zipkin/Jaeger: traces
- Uptime Kuma: uptime

Alternative: ELK

Good log search, but heavier to run and less complete for metrics/tracing without more tooling.

Alternative: Datadog

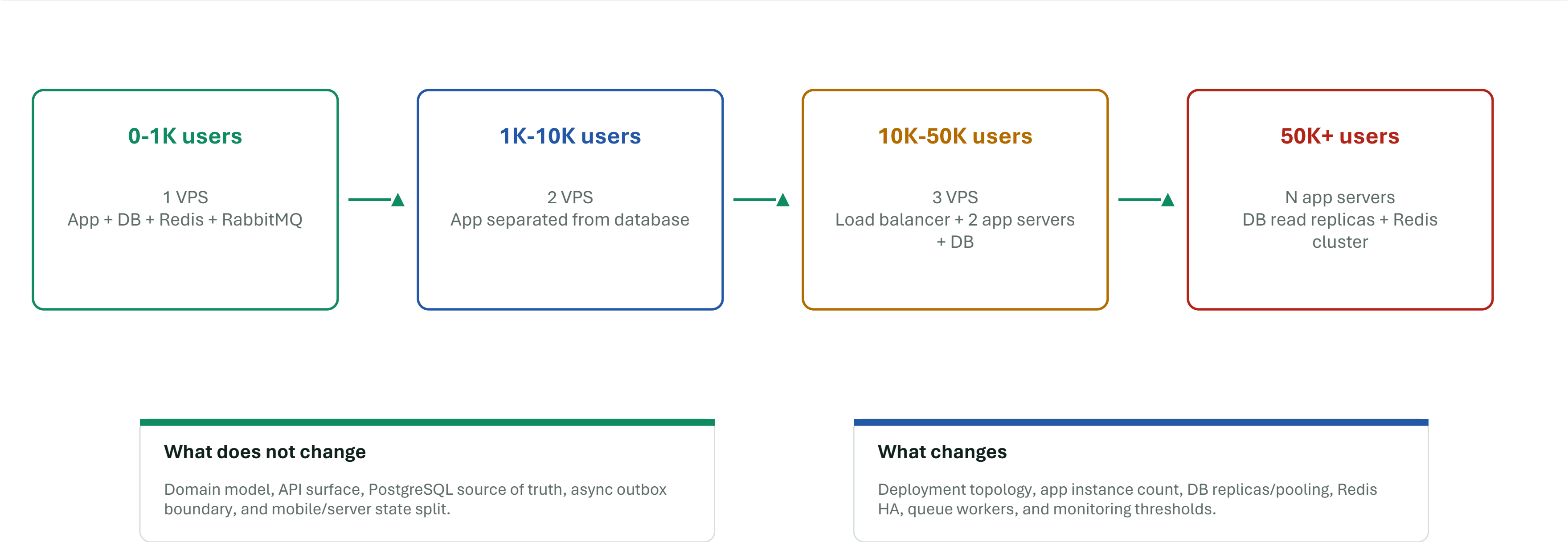
Excellent managed observability, but recurring cost and vendor dependency are higher.

Minimum production checks

- API latency, error rate, JVM memory, DB pool saturation
- Redis/RabbitMQ health and queue depth
- Login, loan, repayment, Polaris sync dashboards
- Structured JSON logs with correlation IDs
- Backup success and restore drill status

Scale infrastructure before changing architecture

The same modular monolith can support multiple traffic stages if app instances, database capacity, and cache/queue topology evolve deliberately.



Approve the stack with named risk controls

The stack is workable if each high-impact risk has an owner and mitigation before production.

Risk	Likelihood	Impact	Owner	Mitigation
Boot 4 compatibility	Med	High	Backend lead	Dependency smoke tests and integration test gate
VPS ops burden	Med	High	DevOps owner	Deployment, backup, restore, patch runbooks
Mobile native modules	Med	Med	Mobile lead	Validate SSL pinning/compressor in Expo build
Security hardening depth	Med	High	Security owner	JWT rotation, refresh lifecycle, audit checklist
Team learning curve	Low	Med	Engineering manager	Code templates, pairing, review standards
Decision gate: no production launch until High-impact risks have named owners and evidence of mitigation.				

Approve with conditions

The proposed stack is coherent for v1 if the team treats operations, security, and compatibility as launch work, not later cleanup.

Recommendation

Approve the supplied stack as the v1 production baseline.

Rationale: best balance of fintech controls, delivery speed, operational simplicity, and staged scalability.

APPROVE WITH CONDITIONS

Conditions for approval

- 1. Validate Spring Boot 4 dependency compatibility
- 2. Write deployment, rollback, backup, and restore runbooks
- 3. Define JWT key rotation and refresh-token lifecycle
- 4. Build dashboards, alerts, logs, traces, and uptime checks
- 5. Validate Expo native-module constraints before mobile release

Next milestone

Backend platform hardening, mobile shell validation, and staging deployment with observability enabled.

Compact version lock reference

Use this slide as the dependency checklist during implementation planning.

Platform	Backend	Data & cache	Mobile	Ops
- Java 21 LTS - Spring Boot 4.0.6 - Spring Framework 7.0.x - Gradle 9.x - Node.js 22 LTS	- JPA / Security / Validation - AMQP / Redis / Flyway / Mail - JWT 0.12.6 - Resilience4j 2.4.0 - Springdoc 2.8.9	- PostgreSQL 17.x - HikariCP 6.x - Hibernate ORM 7.x - Caffeine 3.1.8 - Redis 7.x	- Expo SDK 55.0.x - React Native 0.83 - Expo Router 4.x - TanStack Query 5.x - Zustand 5.x	- Docker 27.x / Compose 2.x - RabbitMQ 4.x - Nginx 1.26.x - Prometheus / Grafana / Loki - Zipkin or Jaeger

Source of truth: user-supplied final tech stack. Treat any implementation drift as an explicit architecture decision.